Running qp

How qp is started, restarted, configured and kept current: the process, the nightly CANSLIM job, the environment, and the operational scripts. Written from `deploy/qp.config.js`, `deploy/qp-restart.sh`, `quant-platform/deploy/`, `chart/server.py` (start-up) and `chart/auth.py`.

Related: the scripts' full list (SCRIPTS.md), where data lives (DATA-MODEL.md).

The process

Name	`qp` under pm2
Command	`python3 -m chart.server --host 0.0.0.0 --port 8765`, from `quant-platform/`
Config	`deploy/qp.config.js`: one instance, auto-restart, **at least 10 s up** to count as started, **at most 10 restarts**, restarted above **500 MB**
Login	`chart/auth.py`: the desk's one password and `td_auth` cookie, checked as pure ASGI so the live WebSocket is covered too; calls from the box itself pass

pm2 is the only supervisor. Never install qp as a systemd service as well: two supervisors fighting for port 8765 restarted it 113,854 times in September. `qp-restart.sh` turns an old `qp-chart` systemd unit off if it finds one.

Start-up

1. **Strategies**, as the server loads: restore them from the JSON copies if the database is empty, sync the seeds, and mark the three finished strategies `ready` where no stage is recorded (STRATEGIES.md).
2. **Wait for the port.** If something holds 8765 it waits up to 60 s, refusing only while a server is actually **listening** (the probe sets `SO_REUSEADDR`, so connections the old process left behind no longer block a restart: KNOWN-ISSUES Q-P).
3. **Mark interrupted backtests**: any run left `running` is marked interrupted. Then it serves.

Restarting it: `deploy/qp-restart.sh <want-sha> [force]`

Called by the deploy. It asks the running qp which build it is (`/api/health` → `build`):

- already this checkout, and not forced: nothing to do;
- otherwise (or forced because `.env` changed): `pm2 restart qp`, or start it from `qp.config.js` if pm2 does not have it, then wait up to 90 s for the new build to answer.

The nightly job: the CANSLIM chain

`quant-platform/deploy/run_daily.py`, run by a **systemd timer** (`qp-daily.timer`, **Monday–Friday 22:30 UTC**, persistent, up to 5 minutes' random delay). A timer runs a job and exits; it supervises nothing, so it does not break the pm2-only rule.

Every link of the chain goes stale on its own clock, so the job rebuilds them all after every close, each step independent (one failing does not stop the rest):

Step	Builds
RS universe top-up	Polygon grouped-daily for the newest sessions (the RS rating's whole-market frame)
market model (M)	`data/oneil-market.json`
industry map (SIC)	SIC classification for new listings and changes
institutional sponsorship (I)	`data/oneil-13f.json`
group ranks (L)	`data/oneil-groups.json`
earnings tables (C, A)	EDGAR facts for the universe, within a time budget

Without it, each piece rebuilds when a page first asks for it, so the first person to open a tool in the morning pays for it, and on a day nobody opens one nothing happens.

Installed by hand, once (the deploy does not install it):

```
sudo cp quant-platform/deploy/qp-daily.{service,timer} /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now qp-daily.timer
systemctl list-timers qp-daily        # when next
journalctl -u qp-daily -f             # watch
```

The units name the `ec2-user` account and its home path. A box without the timer has no nightly chain, which shows only as stale files in the data check (Q-N5).

One-off jobs

Unit / script	Does
`qp-backfill.service` → `run_backfill.py`	one-shot backfills (long; `simple`, not `oneshot`, so no start timeout kills it)
`qp-edgar.service` → `run_edgar.py`	the first full EDGAR pass (thousands of companies)
`qp-13f.service` → `run_13f.py`	builds I on its own
`run_sic.py`	classifies the market into SIC industries and ranks the groups
`run_status.py`	what the CANSLIM chain has and is missing
`run_fields.py AAPL NVDA …`	every CANSLIM field for real tickers, with the reason for each blank

Run by hand from `quant-platform/` with `set -a && . ./.env && set +a` first.

Environment

Variable	Default	Does
`APCA_API_KEY_ID`, `APCA_API_SECRET_KEY`	from `.env`	Alpaca bars and the borrow check (copied from `data/keys.json` by the deploy)
`POLYGON_API_KEY`	—	Polygon bars and the RS universe
`EDGAR_UA`	—	the descriptive User-Agent EDGAR requires
`QP_DB`	`~/.qp/platform.db`	the database
`QP_CACHE_DIR`	`~/.qp-cache`	every cache (each also has its own override)
`QP_CACHE_MAX_MB`, `QP_CACHE_MIN_FREE_MB`	512, 1024	bar cache limits
`QP_DATA_DIR`	`chart/data/`	the operations log
`ONEIL_FUNDAMENTALS_DIR`	`data/oneil/`	EDGAR facts
`SCREENER_SOURCES`, `TOOLS_HOST`, `SCREENER_URL`	from the tool registry, `localhost`	where the screener tools are

---

Watching it

Route	Answers
`GET /api/health`	build (the commit it runs), primitives, feeds with credentials, default feed
`GET /api/feedcheck`	which feeds actually returned data (DATA-FEEDS.md)
`GET /api/datacheck`	every source judged (DATACHECK.md)
`GET /api/oplog`	what qp was asked, what it answered, how long it took (`summary=1` , by `op` or `day`)
`GET /api/screener/sources`	which screener tools are up
`pm2 logs qp --lines 50 --nostream`	the log on the box

Found in this review

#	What
Q-N5	The nightly CANSLIM job is the one scheduler outside pm2, installed by hand with a hard-coded user and path; a fresh box without it silently goes stale
Q-N10	`qp-restart.sh` still says, when qp is down, that “the manager cannot evaluate exit rules or move trailing stops without it”: the order manager no longer exists